GatedV: Self-Gated Values for Transformer Language Models

Li-Ming Yu*
Independent Researcher

November 1, 2026

Abstract

Standard attention forms query (Q), key (K), and value (V) matrices through learned linear projections of input hidden states. Architectures such as GAU and Mega incorporate nonlinear transformations into attention. In this paper, we propose GatedV, a simple attention variant that applies SiLU only to V before aggregation without introducing additional trainable parameters. We evaluate GatedV against a standard decoder-only Transformer using matched parameter counts, paired initializations, and equal training-token budgets. At 2.33M parameters, GatedV reduces mean held-out perplexity from 59.1 to 57.9 (2.0%), with improvements across all five paired seeds. At 26.16M parameters, following an equal-budget learning-rate search, mean held-out perplexity falls from 20.2 to 19.7 (2.4%), with improvements across all three paired seeds. Small-model ablations show that the tested affine rescaling of values does not fully reproduce GatedV's gains. A short interleaved benchmark on NVIDIA DGX Spark measures approximately 0.7% higher full-context forward latency for the larger model, based on synchronized wall-clock timings. These findings suggest that value-only self-gating can improve language modeling performance without increasing parameter count or redesigning the attention-weighting mechanism.

1 Introduction

Standard Transformer blocks combine self-attention with position-wise feed-forward networks [18]. The feed-forward sublayer applies nonlinear transformations to individual token representations, whereas attention mixes information across tokens. Within attention, queries (Q) and keys (K) determine the mixing weights, and values (V) provide the representations to be combined. These three branches are usually formed by linear projections, despite their different roles. An additional nonlinearity on queries or keys changes the weighting calculation; on values, it changes what enters the weighted sum. Moreover, applying a nonlinearity before aggregation is generally not equivalent to applying it afterward.

Nonlinear value representations are already used in several attention architectures. Mega applies SiLU to projected values in a block that also uses moving-average features and additional gates [8]. HYLA introduces nonlinear value computation without additional parameters, together with normalization across attention heads [15]. Neural Attention instead replaces the linear Q/K/V projections with multilayer perceptrons [21]. Gated Attention compares gates at several locations, including the value projection, with its main results using a learned sigmoid gate after scaled dot-product attention [11]. GLU Attention introduces separate content and gate projections and reduces the aggregated value width to match the combined value/output projection-weight count [19]. These approaches differ in both the source of the nonlinearity and the computation surrounding it.

*Email: yuliming1996@gmail.com; ORCID: 0000-0001-7552-8895.

In this paper, we ask whether a nonlinear value transform improves a standard Transformer when the other components are left unchanged. A parameter-free elementwise activation allows the original projection dimensions and block configuration to be retained, while altering only the operation applied to the projected values. We call this variant GatedV (self-gated values). It applies SiLU to the usual value projection V before attention aggregation:

$$\widetilde{V} = \mathrm{SiLU}(V) = V \odot \sigma(V),$$

where σ is the elementwise sigmoid function. The same projected values supply both the content and its gate, so no separate learned gate projection is needed. The query/key projections, softmax weighting rule, and remaining Transformer components are retained. GatedV therefore has exactly the same number of trainable parameters as the baseline, without adjusting the depth or width of either model.

We train baseline and GatedV decoder-only language models from scratch on FineWeb-Edu [10], at 2.33M and 26.16M parameters and with equal training-token budgets within each scale. GatedV reduces mean held-out perplexity by 2.0% and 2.4%, respectively, with improvements in every paired confirmation run. Additional experiments examine activation placement, learning-rate sensitivity, initial value statistics, and computational cost. Whether these improvements persist at larger scales, on other corpora, or on tasks beyond next-token prediction remains to be tested. Code, experiment recipes, and recorded results are available at `https://github.com/yuliming1996/GatedV`.

2 Background and model formulation

2.1 Transformer backbone

Our baseline is a decoder-only Transformer with causal multihead attention [12, 18]. Each block uses RMSNorm before its attention and feed-forward sublayers [20], rotary positional embeddings for queries and keys [17], and a SwiGLU feed-forward transformation [16].

Let $X \in \mathbb{R}^{T\times d}$ denote the hidden states entering a block, where T is the sequence length and d is the model width. We omit the batch dimension and use row-vector notation. The attention sublayer first computes

$$U = \mathrm{RMSNorm}_{\gamma_a}(X), \qquad Q = UW_Q, \quad K = UW_K, \quad V = UW_V, \tag{1}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$. RMSNorm acts on each token independently:

$$\mathrm{RMSNorm}_{\gamma}(x) = \gamma \odot \frac{x}{\sqrt{d^{-1}\sum_{j=1}^{d} x_j^2 + \epsilon}}, \qquad \epsilon = 10^{-6}. \tag{2}$$

The projected tensors are split into H heads of width $d_h = d/H$. For head h, let $\widehat{Q}_h = \mathrm{RoPE}(Q_h)$ and $\widehat{K}_h = \mathrm{RoPE}(K_h)$. We use a rotary base of 10,000. The attention weights and output are

$$A_h = \mathrm{softmax}\left(\frac{\widehat{Q}_h \widehat{K}_h^{\top}}{\sqrt{d_h}} + M\right), \qquad O = \mathrm{Concat}_{h=1}^{H}(A_h V_h), \tag{3}$$

where $M_{ij} = 0$ for $j \le i$ and $M_{ij} = -\infty$ otherwise. Softmax is applied across the key positions in each row. Values enter this aggregation linearly for fixed A_h, although attention as a function of X is nonlinear because the weights themselves depend on the input.

The block then applies the two residual updates

$$X' = X + OW_O, \tag{4}$$

$$Z = \mathrm{RMSNorm}_{\gamma_f}(X'), \tag{5}$$

$$X_{\mathrm{next}} = X' + \big[\,\mathrm{SiLU}(ZW_1) \odot (ZW_2)\big]W_3, \tag{6}$$

where $W_O \in \mathbb{R}^{d\times d}$, $W_1, W_2 \in \mathbb{R}^{d\times d_{\text{ff}}}$, and $W_3 \in \mathbb{R}^{d_{\text{ff}}\times d}$. The SiLU activation is $\text{SiLU}(z) = z\sigma(z)$, with $\sigma(z) = (1+e^{-z})^{-1}$ [5, 14]; both the activation and $\odot$ act elementwise. After L blocks, a final RMSNorm precedes the vocabulary readout, whose weights are tied to the input embedding. All learned linear maps are bias-free. Dropout and additional headwise query/key normalization are disabled.

2.2 Nonlinear projections and attention gates

Nonlinear features can be introduced at several stages of attention, beginning with the projections that construct Q, K, and V. Neural Attention replaces these linear maps with two-layer networks containing LayerNorm and ReLU [21]. This increases the projection parameter count. Its ReLU ablation retains the expanded key/value projections and LayerNorm, testing the activation within the modified network rather than comparing with a parameter-matched standard Transformer.

Nonlinear values also appear in architectures that redesign the attention block more broadly. The Gated Attention Unit (GAU) combines nonlinear values and shared query/key features with squared-ReLU attention and multiplicative gating in a unified attention and feed-forward layer [6]. Mega builds on this construction, deriving shared query/key features from exponential moving averages and adding reset and update gates [8]. Its value branch applies SiLU directly to the projected inputs before aggregation. This provides a direct precedent for value-side SiLU, although the surrounding computation differs from conventional multihead attention.

GLU Attention retains softmax aggregation and uses two independently projected branches, $\widetilde{V} = (UW_C) \odot \text{SiLU}(UW_G)$ [19]. For model width d, it sets the combined value width across heads to $m = 2d/3$. Ignoring biases, the two $d \times m$ input projections and the $m \times d$ output projection then contain $3dm = 2d^2$ weights, matching the standard value and output projections. Thus its parameter matching involves narrower values and a separate learned gate. GatedV instead preserves the original value width and projection matrices, applying SiLU directly to a single projected value tensor.

Other formulations make the nonlinear transformation depend on how values are combined. HYLA applies ReLU to an attention-weighted combination of value features across heads for each query–key pair. It introduces no additional parameters and uses RMS normalization across heads instead of key-wise softmax [15]. LASER retains softmax weights but combines an exponential value transform with a logarithm after aggregation, giving $\log(A_h \exp(V_h))$ [4]. These operations are not generally equivalent to applying an activation independently to each projected value and then taking the usual weighted sum.

Gate placement has also been studied while retaining softmax attention. Gated Attention compares learned gates on Q/K/V and at the attention output, with its main configuration using a head-specific sigmoid gate after scaled dot-product attention [11]. Its G2 gate uses a separate input projection to compute a sigmoid gate on the values before aggregation. It also evaluates SiLU applied directly to the attention output without additional parameters (their Table 3, row 6). Two distinctions are relevant here: whether the gate uses a separate learned projection, and whether it acts before or after aggregation. We examine self-gating of the projected values before aggregation, retaining the original projections and the softmax weighting rule.

2.3 GatedV

GatedV replaces the projected values in Equation 1 with

$$\widetilde{V} = \phi(V) = V \odot \sigma(V), \qquad \phi = \text{SiLU}\,. \tag{7}$$

The Q/K/V projections are implemented as one fused linear map. After splitting its output into three tensors, we apply ϕ to V before reshaping into heads. The per-head comparison is

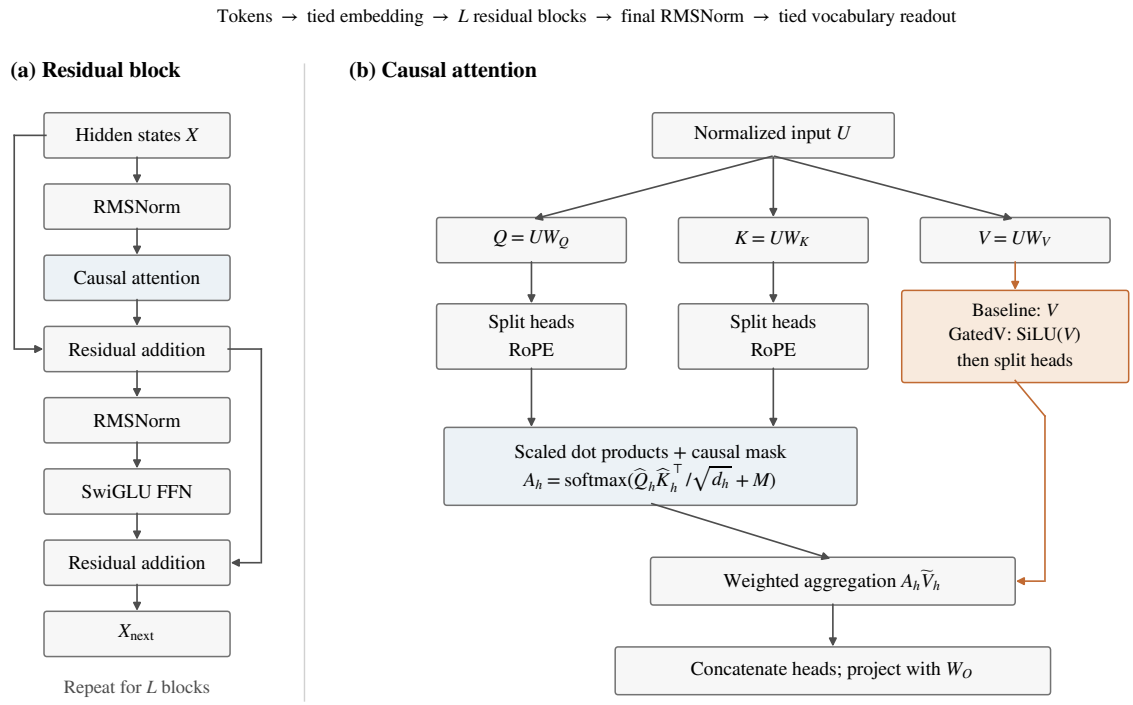


Figure 1: Transformer block and GatedV value transformation. The baseline aggregates the projected values V, whereas GatedV applies $\mathrm{SiLU}(V)$ before aggregation. The value box shows the two alternatives; each model uses one.

therefore

$$Y_h^{\text{base}} = A_h V_h, \qquad Y_h^{\text{GatedV}} = A_h \phi(V_h). \tag{8}$$

Figure 1 shows where the value activation is applied.

The gate is determined by the same projected value that it multiplies. It requires no separate projection such as W_G in $(UW_V) \odot \sigma(UW_G)$, the two-projection form of a gated linear unit [2]. Placement also matters: in general,

$$A_h \phi(V_h) \neq \phi(A_h V_h). \tag{9}$$

The left-hand side transforms each source token's value before mixing it with other values; the right-hand side transforms the resulting mixture.

For a fixed normalized input U and fixed query/key projection weights, GatedV leaves A_h unchanged. No trainable tensors are added or resized. The depth, hidden widths, head structure, normalization, positional encoding, and feed-forward blocks remain as defined above, so paired baseline and GatedV models can start from identical weights. The activation adds $O(Td)$ elementwise work per block.

For the activation-placement experiments, we also apply ϕ to queries alone or to all three of Q, K, and V. Query and key activations are applied before RoPE. These variants retain the original depth and projection dimensions.

Table 1: Main confirmation settings. Both models use a vocabulary of 8,192, context length 256, and batch size 8. Target exposures count sampled prediction targets, including repetitions.

Setting	Small	Medium
Layers / model width	6 / 128	12 / 384
Heads / head width	4 / 32	6 / 64
SwiGLU hidden width	384	1,152
Trainable parameters	2,328,192	26,158,464
Training-corpus tokens	40,001,310	512,000,068
Training-corpus documents	27,830	212,052
Updates per run	32,768	327,680
Target exposures per run	67,108,864	671,088,640
Warmup updates	512	5,120
Confirmation peak learning rate	0.0048	0.0003
Paired confirmation seeds	5	3

3 Experimental design

3.1 Models, data, and training

We compare baseline and GatedV models at two sizes (Table 1), with identical initial weights and sampled training windows. Within each pair, only the value activation differs.

Training uses FineWeb-Edu text [10] and a fixed 8,192-entry byte-level BPE tokenizer [13] trained on WikiText training data [9]. Each training sequence is sampled from a single document and contains 256 input tokens with their next-token targets. The small confirmation held-out set and the medium development and held-out sets each contain 2,048 documents from distinct domain groups, with the first eligible 256-target window per document.

We train with AdamW [7], using $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, matrix weight decay 0.1, no decay on normalization gains, and gradient-norm clipping at 1. After warmup, the learning rate follows a cosine schedule to 10% of its peak. We evaluate final fixed-budget checkpoints. Training uses FP32. We initialize projection and embedding matrices from a Gaussian distribution with fan-in scaling, with an additional factor of $1/\sqrt{2L}$ for attention output and feed-forward down-projections, where L is the number of Transformer layers.

3.2 Learning-rate selection and controls

The small-model confirmation compares baseline and GatedV at a shared peak learning rate of 0.0048 over five paired seeds. We fixed this rate after a 4,096-update development screen in which both models achieved their lowest NLL at the highest rate tested.

After the original medium-model confirmation at 0.0006, we select each architecture's learning rate from $\{0.00001, 0.0001, 0.0003, 0.0006\}$ by the lowest final development NLL, using one shared selection seed (20270201) and equal training budgets; ties favor the lower rate. Both select 0.0003 (Table 2). We evaluate these configurations with the original three paired seeds (20270211–20270213), distinct from the selection seed, on the previously used medium held-out set. The original 0.0006 confirmation is retained in Appendix A. A separate development-set comparison evaluates value gates at $\{0.0006, 0.0003, 0.0001\}$ (Section 5.1).

Small-model ablations compare activation placement and learning rates using three paired seeds and 32,768 updates per run (Table 4). We also test whether fixed adjustments to the initial value statistics can reproduce GatedV's gain. Affine match rescales and shifts the baseline values to match the mean and variance of SiLU-transformed values; Moment-matched SiLU applies the reverse adjustment to SiLU-transformed values, matching the original value statistics. For

Table 2: Medium-model learning-rate selection: final development PPL at the shared selection seed 20270201, with 327,680 updates per run. Both architectures select 0.0003.

Variant	LR 0.00001	LR 0.0001	LR 0.0003	LR 0.0006
Baseline	36.853	20.938	20.558	21.558
GatedV	37.451	20.632	20.062	21.005

each seed and layer, we estimate (μ_V, σ_V) and (μ_ϕ, σ_ϕ) from 128 training windows at baseline initialization and define

$$V_{\text{aff}} = \frac{\sigma_\phi}{\sigma_V}(V - \mu_V) + \mu_\phi, \tag{10}$$

$$V_{\text{mm}} = \frac{\sigma_V}{\sigma_\phi}(\phi(V) - \mu_\phi) + \mu_V. \tag{11}$$

The coefficients remain fixed throughout training. At peak learning rates 0.0024 and 0.0048, with three paired seeds and 32,768 updates per run, both controls have higher mean development PPL than GatedV at the matching rate, by 1.77–2.82% (see Table 5 and Section 4.2). Thus these fixed moment adjustments do not reproduce GatedV's full gain.

3.3 Evaluation and uncertainty

For seed s, document loss sums $\ell_{s,i}$, and target counts n_i, we report token-averaged negative log-likelihood and perplexity as

$$L_s = \frac{\sum_i \ell_{s,i}}{\sum_i n_i}, \qquad P_s = \exp(L_s), \qquad \Delta_P = 100\left(\frac{\overline{P}_{\text{GatedV}}}{\overline{P}_{\text{base}}} - 1\right). \tag{12}$$

$\overline{P}$ is the arithmetic mean of per-seed perplexities, not the exponential of mean NLL. Negative Δ_P denotes improvement. For multi-seed PPL results, $\pm$ denotes the sample standard deviation across training seeds.

To estimate uncertainty from evaluation texts in the main held-out comparisons [1, 3], we keep the trained models fixed and average each document's summed loss difference across seed pairs. We then draw 2,000 bootstrap samples of the documents, with replacement and at the original sample size. For each sample, we recompute the token-averaged NLL difference; the 2.5th and 97.5th percentiles of these estimates define the 95% bootstrap interval (see Section 4.1).

4 Results

4.1 Held-out language modeling

Table 3 summarizes the two confirmations. At 2.33M parameters, mean held-out PPL decreases by 2.03%, with all five pairs improving. Individual reductions range from 0.33% to 3.74%. At 26.16M parameters and the development-selected learning rate of 0.0003, all three confirmation pairs improve, reducing mean held-out PPL by 2.40%. Individual reductions range from 2.22% to 2.66%.

The mean GatedV-minus-baseline NLL differences are -0.020591 and -0.024288 for the small and medium models, respectively. The corresponding 95% document-bootstrap intervals, with the trained models held fixed, are $[-0.021869, -0.019356]$ and $[-0.025507, -0.023153]$. After averaging NLL across the three seeds for each model, we find that GatedV has lower NLL on 1,672 of the 2,048 medium-model held-out windows (81.6%).

Table 3: Held-out confirmation results. Values are arithmetic mean PPL ± sample SD. The studies use different held-out sets and training configurations; the medium model uses peak LR 0.0003.

Model size	Baseline	GatedV	Δ_P	Improved pairs
2.33M	59.118 ± 0.530	57.916 ± 0.782	-2.03%	5/5
26.16M	20.233 ± 0.068	19.748 ± 0.047	-2.40%	3/3

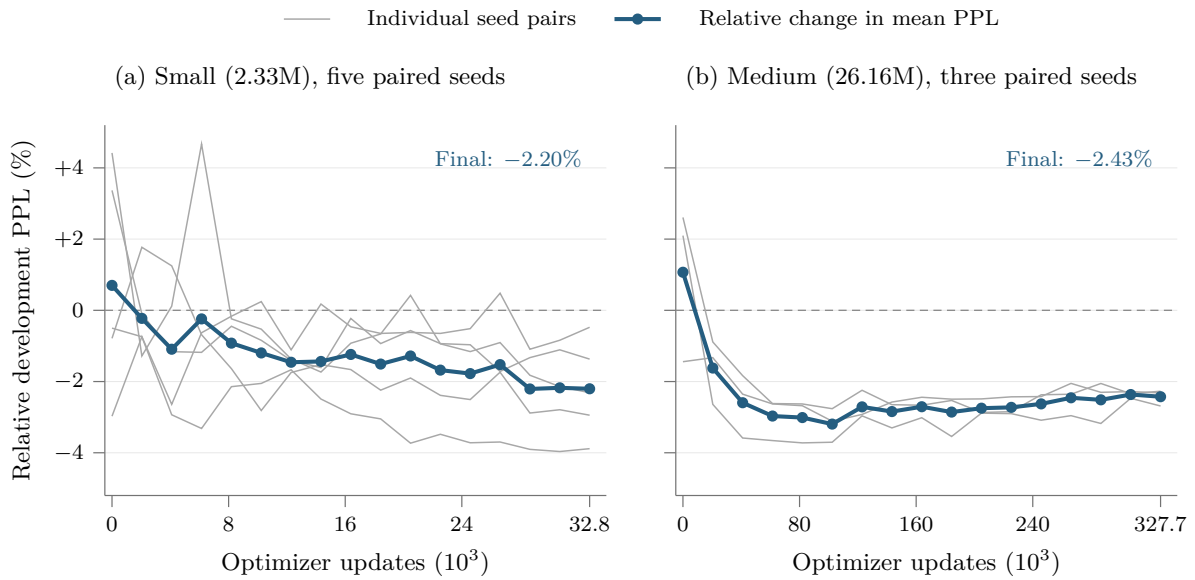


Figure 2: Relative development PPL, $\Delta\text{PPL}/P_{\text{base}}$, shown as a percentage; ΔPPL denotes GatedV minus baseline. Thin gray curves show individual seed pairs; the thick blue curve uses the arithmetic mean PPL of each model across seeds. Negative values favor GatedV.

Figure 2 shows the relative change in development PPL throughout training, including initialization. At each recorded update, we exponentiate each run's development NLL and then average PPL across seeds; the mean curve uses the ratio of these arithmetic means, as in Equation 12. The positive initial Δ_P in both panels may reflect the different activation statistics produced by SiLU under the shared Gaussian fan-in initialization. Figure 3 shows absolute PPL over the final quarter of training. Final development means are 58.716 versus 57.422 for the small model and 20.640 versus 20.139 for the medium model, corresponding to reductions of 2.20% and 2.43%.

4.2 Activation placement and learning rate

Table 4 summarizes the small-model activation-placement and learning-rate comparisons. At the same six-layer depth, value-only SiLU improves all three pairs. Query-only SiLU worsens all three, while joint Q/K/V SiLU slightly lowers mean PPL, with improvements in two of the three pairs.

In the separate 32,768-update exploratory small-model learning-rate grid, both architectures select the interior rate 0.0024 by mean final development NLL. GatedV improves mean PPL by 3.08% there and improves every pair. Its mean is favorable at all four rates, but one pair worsens at each of 0.0012 and 0.0096. The medium-model comparison at lower learning rates is reported in Table 7.

Relative to baseline, Affine match changes mean development PPL by -1.12% at LR 0.0024

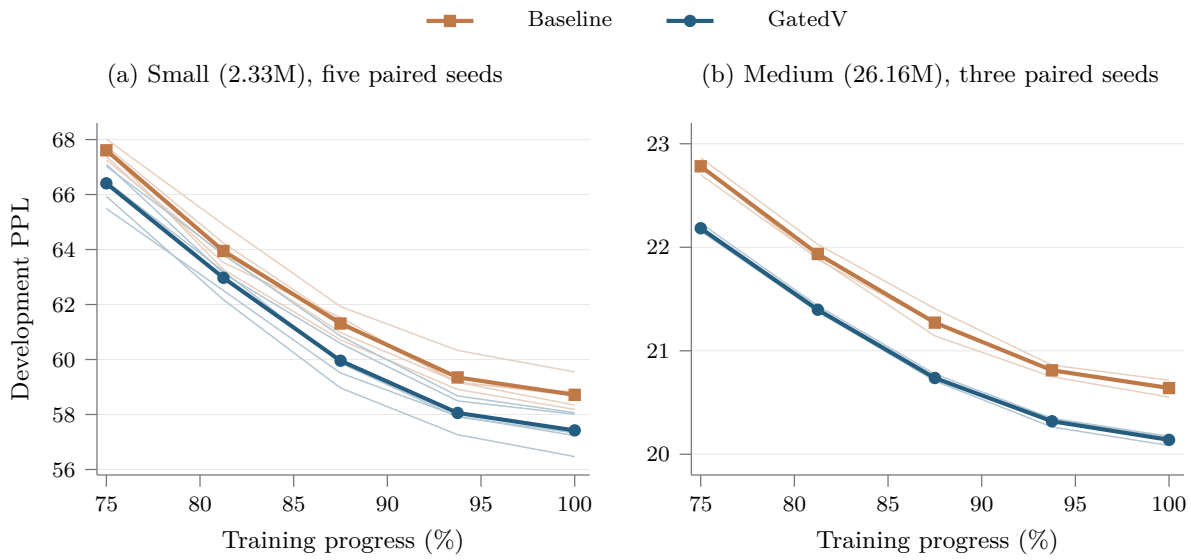


Figure 3: Development PPL over the final 25% of training. Copper denotes baseline and blue denotes GatedV. Thin light curves show individual runs; thick curves show arithmetic means across seeds. Held-out endpoints are reported separately in Table 3.

Table 4: Development-set comparisons. The two groups of rows use different sets of three paired seeds. Δ_P compares the variant with the baseline.

Variant or peak LR	Baseline PPL	Variant PPL	Δ_P	Improved pairs
Activation placement; LR = 0.0048				
Q-only	58.580 ± 0.557	59.522 ± 0.337	+1.61%	0/3
GatedV	58.580 ± 0.557	58.039 ± 0.525	−0.92%	3/3
All Q/K/V	58.580 ± 0.557	58.458 ± 0.485	−0.21%	2/3
Small-model LR study; variant is GatedV				
0.0012	56.441 ± 0.548	55.269 ± 1.120	−2.08%	2/3
0.0024	56.180 ± 0.462	54.450 ± 0.140	−3.08%	3/3
0.0048	58.770 ± 0.983	57.384 ± 0.168	−2.36%	3/3
0.0096	65.168 ± 1.333	64.063 ± 0.442	−1.70%	2/3

and +0.26% at 0.0048 (Table 5). Moment-matched SiLU uses a fixed shift and rescaling to match SiLU outputs to the initial baseline value mean and variance; its PPL reductions are 0.35% and 0.63%, respectively. Both controls have higher PPL than GatedV in every pair.

4.3 Computational cost

We benchmark the six medium-model checkpoints from the 0.0006 confirmation on NVIDIA DGX Spark with fixed synthetic inputs, batch size 8, and context length 256. The test takes about 97 seconds on a shared host, alternating models over seven rounds with three warmups and ten timed calls per mode. Wall-clock timings use CUDA synchronization. We compute paired time ratios from median call times, take the median across rounds for each seed, and average across seeds.

Measured latency increases by 0.70% for full-context forward passes and 1.14% for training steps with GatedV (Table 6). Across the 21 paired comparisons, the relative change in forward latency for GatedV versus baseline ranges from −1.15% to +2.57%. Median peak CUDA allocated memory rises from 1.7525 to 1.8569 GiB (5.96%).

Table 5: Final development PPL for fixed moment controls, with three paired seeds. Relative changes compare each control with baseline or GatedV at the same peak LR.

Control	Peak LR	PPL ± SD	vs baseline	vs GatedV
Affine match	0.0024	55.551 ± 0.498	−1.12%	+2.02%
Moment-matched SiLU	0.0024	55.983 ± 0.787	−0.35%	+2.82%
Affine match	0.0048	58.920 ± 0.298	+0.26%	+2.68%
Moment-matched SiLU	0.0048	58.399 ± 0.540	−0.63%	+1.77%

Table 6: Medium-model time ratios relative to baseline and median peak CUDA allocated memory over the forward and training-step measurements combined.

Measure	Baseline	GatedV
Full-context forward time ratio	1.000000	1.007003
Training-step time ratio	1.000000	1.011433
CUDA peak allocated (GiB)	1.7525	1.8569

5 Discussion

GatedV improves held-out language modeling in both tested configurations without adding parameters. In the medium model, its advantage narrows from 3.66% at peak LR 0.0006 to 2.40% at the development-selected rate of 0.0003: lowering the rate improves the baseline more, while GatedV remains better in all three pairs (Appendix A). The branch comparisons show that this benefit is not shared by every activation placement.

5.1 Comparison with GLU and tanh

GatedV uses a single value projection with sigmoid self-gating. We compare it with two-projection GLU value gates and a parameter-free tanh alternative in the medium model (Table 7). The GLU controls use $(UW_C) \odot \mathrm{SiLU}(UW_G)$ before aggregation, following GLU Attention [19]. The full-width control retains 384 value dimensions, adding a separate projection; the reduced-width control uses 258 dimensions, the nearest six-head-compatible total width to $2d/3$, to approximate the baseline parameter count. The tanh control applies $V \odot \tanh(V)$ at the original width. All runs process the same sampled training windows and share identical initial weights for corresponding matrices with matching shapes.

Table 7: Exploratory medium-model value-gate comparison: final development PPL with one shared seed (20270201), reused development data, and 327,680 updates per run.

Variant	Parameters (M)	LR 0.0006	LR 0.0003	LR 0.0001
Baseline	26.158	21.558	20.558	20.938
GatedV	26.158	21.005	20.062	20.632
Full-width GLU	27.928	21.197	20.309	20.930
Reduced-width GLU	26.186	21.522	20.657	21.382
$V \odot \tanh(V)$	26.158	21.337	20.325	20.986

All five variants achieve their lowest final development PPL at the interior rate 0.0003. The separate GLU gate does not improve on GatedV in this grid, either at full width with more parameters or at reduced width with a similar parameter count. At the selected rate, the tanh control also improves on the baseline, but GatedV remains 1.29% lower in PPL (2.42% below the baseline). On this shared seed, GatedV has the lowest development PPL at each of the three learning rates.

5.2 Limitations

Our experiments are limited to models below 28M parameters on FineWeb-Edu, using 256-token contexts and next-token prediction. Larger models, longer contexts, other corpora, and downstream tasks remain untested. The GLU and tanh comparisons use a single seed, and cost measurements cover only our FP32 implementation.

6 Conclusion

Applying SiLU to values before aggregation improves held-out perplexity by 2.03% and 2.40% in the two tested Transformer configurations, with all paired confirmation runs improving. The change preserves parameter count, depth, and width. The exploratory controls show that the gain is not shared by every activation placement or learning rate, and that the tested initial-moment adjustments do not reproduce it fully. The measured runtime increase is small in this implementation, but its precise magnitude and the extension to larger models, other tasks, and optimized inference remain unresolved.

A Medium-model learning-rate comparison

The original medium-model search used $\{0.0006, 0.0012, 0.0024\}$ and selected its lower boundary, 0.0006, for both architectures. Table 8 retains that confirmation alongside the 0.0003 results. Both rates use the same three seed pairs, matched initial weights and training-window schedules, 327,680 updates per run, and the same development and held-out sets.

Table 8: Medium-model confirmation at two peak learning rates. PPL is the arithmetic mean ± sample SD across seeds 20270211–20270213; Δ_P compares GatedV with baseline within each row.

Evaluation set	Peak LR	Baseline PPL	GatedV PPL	Δ_P
Development	0.0006	21.804 ± 0.182	21.039 ± 0.045	-3.51%
Development	0.0003	20.640 ± 0.082	20.139 ± 0.047	-2.43%
Held-out	0.0006	21.384 ± 0.166	20.600 ± 0.018	-3.66%
Held-out	0.0003	20.233 ± 0.068	19.748 ± 0.047	-2.40%

Reducing the peak learning rate from 0.0006 to 0.0003 lowers mean held-out PPL by 5.38% for baseline and 4.14% for GatedV. All three pairs favor GatedV at each rate. We report the rates separately because they measure the advantage under different optimization settings.

B Value activation during training and inference

We held the weights of each final medium-model checkpoint (peak LR 0.0003) fixed and evaluated it on the same 2,048 held-out windows with either linear values or SiLU applied to the values in every attention layer. Switching the value transformation at inference raised PPL for all six checkpoints, most sharply when we removed SiLU from GatedV (Table 9).

Table 9: Held-out PPL (mean and sample standard deviation across three paired seeds).

Training	Linear at inference	SiLU at inference
Linear (baseline)	20.233 ± 0.068	102.565 ± 31.053
SiLU (GatedV)	$4{,}300.232 \pm 899.037$	19.748 ± 0.047

References

[1] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems*, 3:747–769, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.

[2] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 933–941. PMLR, 2017. URL https://proceedings.mlr.press/v70/dauphin17a.html.

[3] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1128. URL https://aclanthology.org/P18-1128/.

[4] S. S. Duvvuri and I. S. Dhillon. LASER: Attention with exponential transformation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 14902–14917. PMLR, 2025. URL https://proceedings.mlr.press/v267/duvvuri25a.html.

[5] S. Elfwing, E. Uchibe, and K. Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017. URL https://arxiv.org/abs/1702.03118.

[6] W. Hua, Z. Dai, H. Liu, and Q. Le. Transformer quality in linear time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9099–9117. PMLR, 2022. URL https://proceedings.mlr.press/v162/hua22a.html.

[7] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[8] X. Ma, C. Zhou, X. Kong, J. He, L. Gui, G. Neubig, J. May, and L. Zettlemoyer. Mega: Moving average equipped gated attention. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2209.10655. First preprint in 2022; equations checked in arXiv:2209.10655v3.

[9] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

[10] G. Penedo, H. Kydlíček, L. Ben allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-0970. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/370

df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html. Datasets and Benchmarks Track.

[11] Z. Qiu, Z. Wang, B. Zheng, Z. Huang, K. Wen, S. Yang, R. Men, L. Yu, F. Huang, S. Huang, D. Liu, J. Zhou, and J. Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-3345. URL https://papers.neurips.cc/paper_files/paper/2025/hash/904e89bb4e632e75fb47f093b620b257-Abstract-Conference.html.

[12] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training, 2018. URL https://www.mikecaptain.com/resources/pdf/GPT-1.pdf.

[13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[14] P. Ramachandran, B. Zoph, and Q. V. Le. Searching for activation functions, 2017. URL https://arxiv.org/abs/1710.05941.

[15] S. Schug, S. Kobayashi, Y. Akram, J. Sacramento, and R. Pascanu. Attention as a hypernetwork. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2406.05816. First preprint in 2024; final arXiv:2406.05816v4 checked.

[16] N. Shazeer. GLU variants improve transformer, 2020. URL https://arxiv.org/abs/2002.05202.

[17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021. URL https://arxiv.org/abs/2104.09864.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[19] Z. Wang. GLU attention improve transformer, 2025. URL https://arxiv.org/abs/2507.00022.

[20] B. Zhang and R. Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html.

[21] M. Zhang. Neural attention: Enhancing QKV calculation in self-attention mechanism with neural networks, 2023. URL https://arxiv.org/abs/2310.11398.